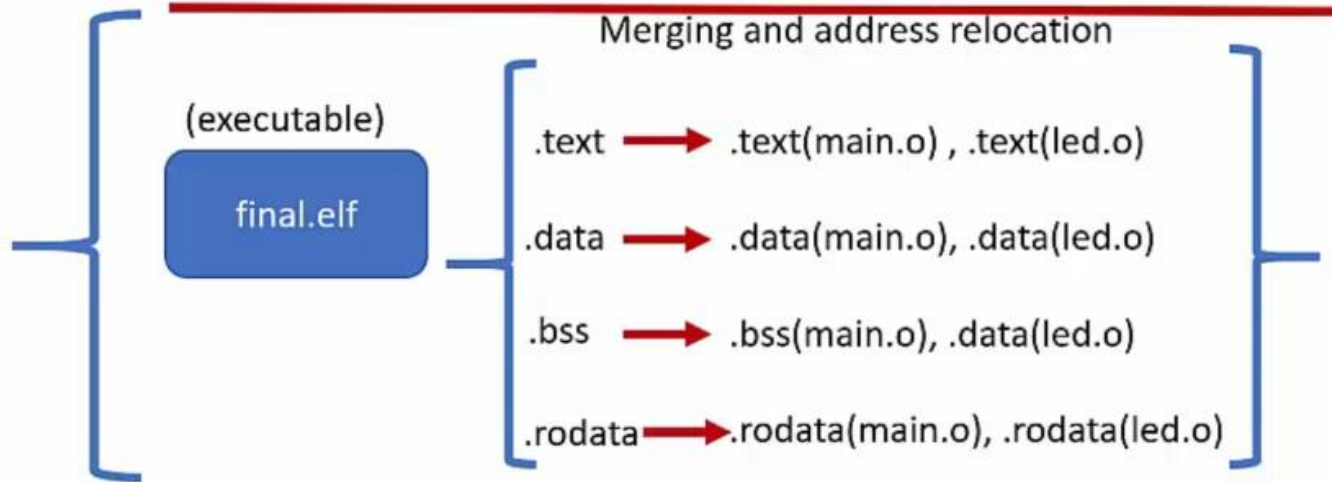


Done by linker and locator
With the help of linker script
provided by you



```
main.o:main.c
```

```
$(CC) $(CFLAGS) main.c -o main.o
```

$\$^{\wedge}$ → denotes dependency

$\$@$ → denotes target

$\$(CC)$ $\$(CFLAGS)$ $\$^{\wedge}$ -o $\$@$

$\$(CC)$ $\$(CFLAGS)$ -o $\$@$ $\$^{\wedge}$

```
#include<stdint.h>
```

```
uint32_t d_count = 300000; data
```

```
int g_un_data1;| bss  
int g_un_data2 = 0; bss  
int g_i_data = 0x55; //.data  
static int g_un_s_data; bss  
static int g_i_s_data = 0x44; //.data
```

```
int main(void){
```

```
printf("data = %d,%d,%d,%d,%d\n",g_un_data1,g_un_data2,g_i_data,g_un_s_data,g_i_s_data);
```

```
int l_un_data; stack  
int l_i_data = 0x98; data  
static int l_un_s_data; bss  
static int l_i_s_data = 0x24; //.data
```

```
printf("sum = %d\n",l_un_data+l_i_data+l_un_s_data+l_i_s_data);
```

```
}
```

1. 起始點
2. 非必要, 但debug .elf會用到

ENTRY command

- This command is used to set the "**Entry point address**" information in the header of final elf file generated
- In our case, "Reset_Handler" is the entry point into the application. The first piece of code that executes right after the processor reset.
- The debugger uses this information to locate the first function to execute.
- Not a mandatory command to use, but required when you debug the elf file using the debugger (GDB)
- Syntax : Entry(_symbol_name_)
Entry(Reset_Handler)

描述 FLASH和SRAM

Memory command

- This command allows you to describe the different memories present in the target and their start address and size information
- The linker uses information mentioned in this command to assign addresses to merged sections
- The information is given under this command also helps the linker to calculate total code and data memory consumed so far and throw an error message if data, code, heap or stack areas cannot fit into available size
- By using memory command, you can fine-tune various memories available in your target and allow different sections to occupy different memory areas
- Typically one linker script has one memory command

Memory command

Syntax :

MEMORY

{
}
}

name (attr) : ORIGIN = origin, LENGTH = len

Defines name of the memory region which will be later referenced by other parts of the linker script

defines origin address of the memory region

Defines the length information

Attribute letter	Meaning
R	Read-only sections
W	Read and write sections
X	Sections containing executable code.
A	Allocated sections
I	Initialized sections.
L	Same as 'I'
!	Invert the sense of any of the following attributes.

Sections command

- SECTIONS command is used to create different output sections in the final elf executable generated.
- Important command by which you can instruct the linker how to merge the input sections to yield an output section
- This command also controls the order in which different output sections appear in the elf file generated.
- By using this command, you also mention the placement of a section in a memory region. For example, you instruct the linker to place the `.text` section in the `FLASH memory region`, which is described by the MEMORY command.

```
/* Sections */
```

```
SECTIONS
```

```
{
```

```
/* This section should include .text section of all input files */
```

```
• .text :
```

```
{
```

```
    //merge all .isr_vector section of all input files
```

```
    //merge all .text section of all input files
```

```
    //merge all .rodata section of all input files
```

```
}>(vma) AT>(lma)
```

```
/* This section should include .data section of all input files */
```

```
.data :
```

```
{
```

```
    //here merge all .data section of all input files
```

```
}>(vma) AT>(lma)
```

```
}
```

```
3 MEMORY
```

```
4 {
```

```
5     FLASH(rx) : ORIGIN=0x08000000, LENGTH=1024K
```

```
6     SRAM(rwx) : ORIGIN=0x20000000, LENGTH=128K
```

```
7 }
```

```
8
```

```
9 SECTIONS
```

```
0 {
```

```
1     .text :
```

```
2     {
```

```
3         *(.isr_vector)
```

```
4         *(.text)
```

```
5         *(.rodata)
```

```
6     }> FLASH
```

```
7
```

```
8     .data
```

```
9     {
```

```
0         *(.data)
```

```
1     }> SRAM AT> FLASH
```

```
2
```

```
3     .bss
```

```
4     {
```

```
5         *(.bss) I
```

```
6     }> SRAM
```

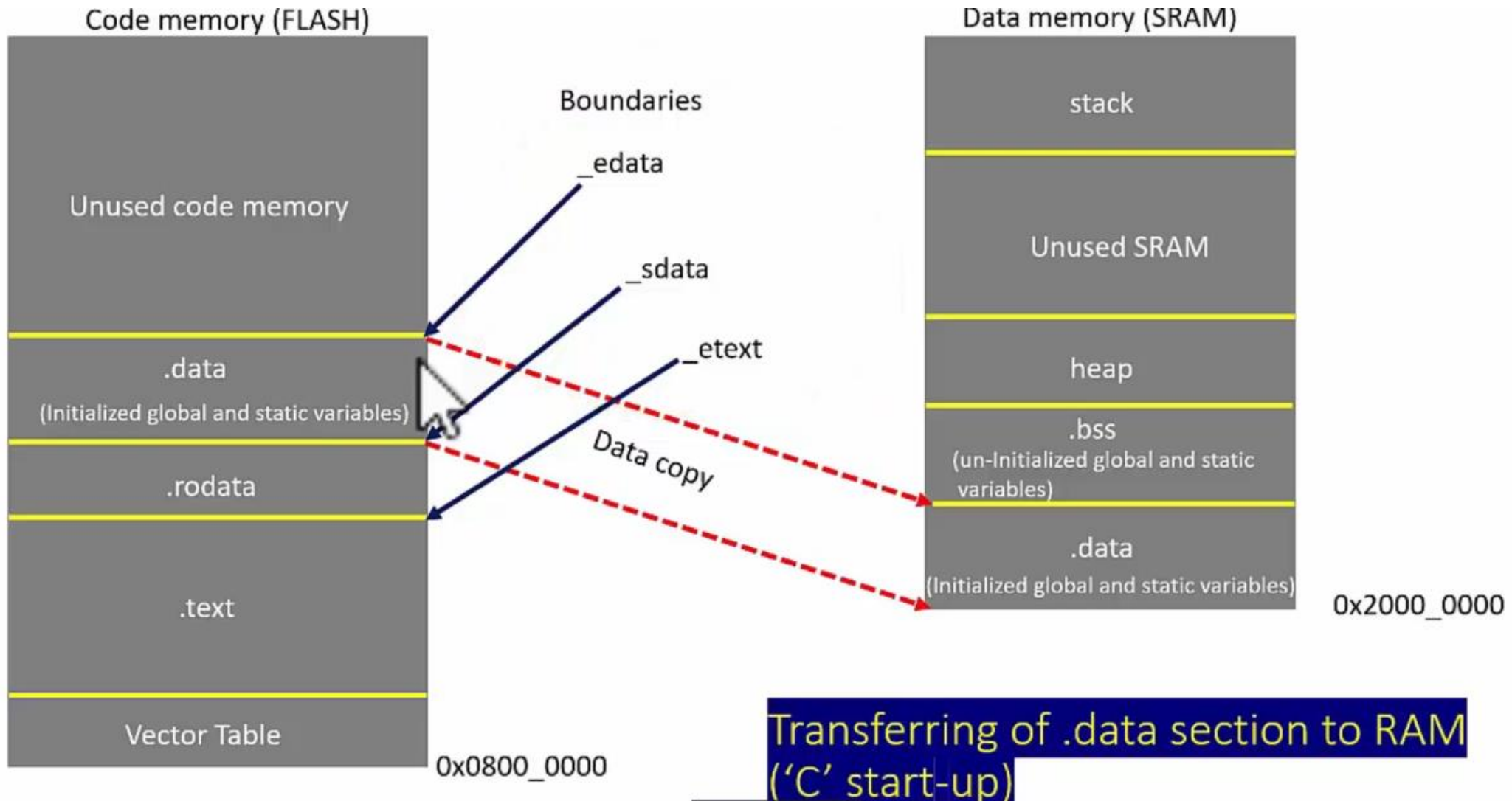
```
7
```


Location counter

1. 因為要把在FLASH裡的.data複製到SRAM裡的.data, 所以要知道section的邊界
2. Linker script會自動計算其大小
3. 在linker script是一個點
4. Location counter追蹤VMA

Location counter (.)

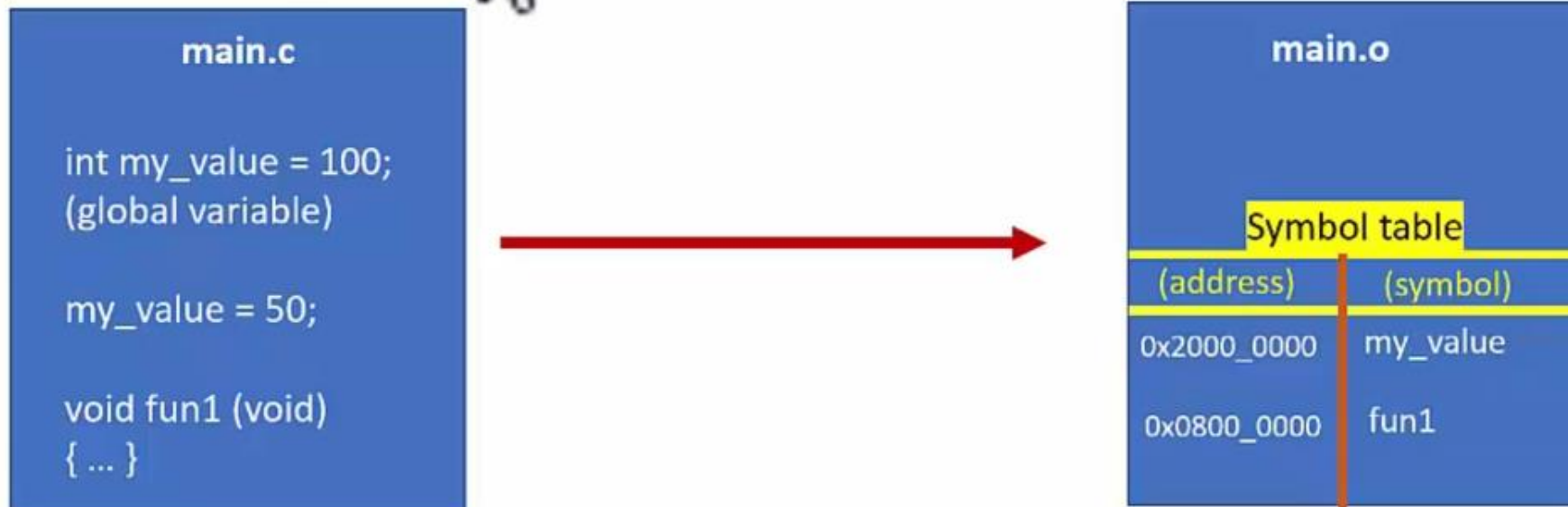
- This is a special linker symbol denoted by a dot ‘.’
- This symbol is called “location counter” since linker automatically updates this symbol with location(address) information
- You can use this symbol inside the linker script to track and define boundaries of various sections
- You can also set location counter to any specific value while writing linker script
- Location counter should appear only inside the SECTIONS command
- The location counter is incremented by the size of the output section



Symbol: it is a name given by address

Linker script symbol

- A symbol is the name of an address
- A symbol declaration is not equivalent to a variable declaration what you do in your 'C' application



```

1 ENTRY(Reset_Handler)
2
3 MEMORY
4 {
5     FLASH(rx):ORIGIN=0x08000000,LENGTH=1024K
6     SRAM(rwx):ORIGIN=0x20000000,LENGTH=128K
7 }
8
9 __max_heap_size = 0x400; /* A symbol declaration . Not a variable !!! */
10 __max_stack_size = 0x200; /* A symbol declaration . Not a variable !!! */
11
12 SECTIONS
13 {
14     .text :
15     {
16         • *.isr_vector)
17         *.text)
18         *.rodata)
19         end of text = 0; /* Store the updated location counter value in to a symbol 'end_of_test' */
20     }> FLASH
21
22     .data
23     {
24         start_of_data = 0x20000000; /* Assign a value to a symbol 'start_of_data' */
25         *.data)
26     }> SRAM AT> FLASH
27

```

Compiler不知道, 因為是在linker script宣告

Linker script 假設起始地址為0x08000000

這個點是location counter

Location counter 追蹤 VMA

```
}  
  
SECTIONS  
{  
  .text :  
  {  
    * (.isr_vector)  
    * (.text)  
    * (.rodata)  
    _etext = .;  
  } > FLASH  
  
  .data  
  {  
    sdata = .;  
    * (.data)  
  } > SRAM AT> FLASH  
  
  .bss  
  {  
    * (.bss)  
  } > SRAM
```

Note:

Location counter always tracks VMA of the section in which it is being used



Here, the location counter resets to start of VMA of this section.

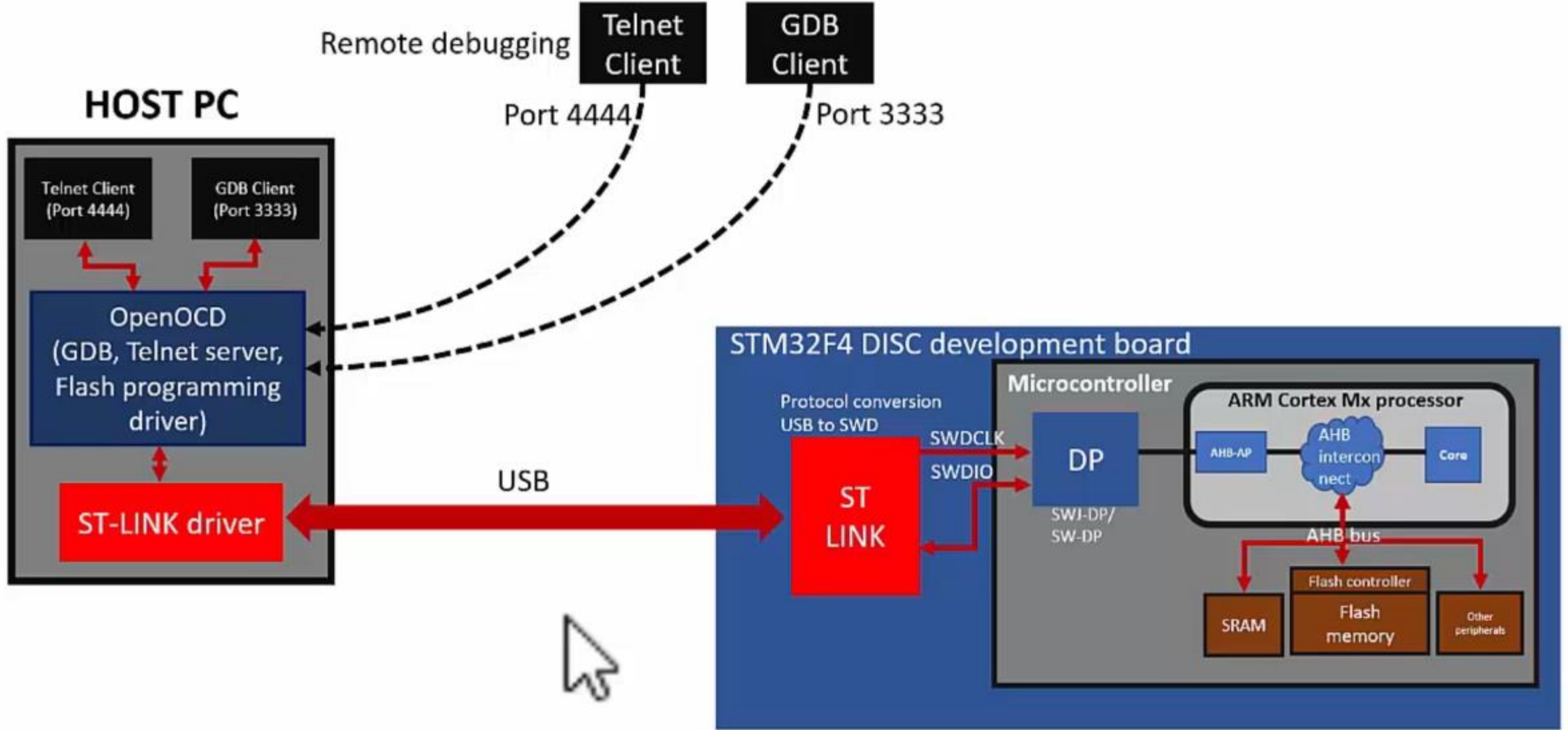
SECTIONS

```
{
    .text :
    {
        *(.isr_vector)
        *(.text)
        *(.rodata)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    .data :
    {
        _sdata = .;
        *(.data)
        _edata = .;
    } > SRAM AT> FLASH

    .bss :
    {
        _sbss = .;
        *(.bss)
        _ebss = .;
    } > SRAM
}
```

使location counter變成word aligned



```

bo@harry MINGW64 /c/Users/bo/Documents/bare_metal_embedded_system
$ make load
openocd -f board/stm32f4discovery.cfg
xPack Open On-Chip Debugger 0.12.0+dev-01850-geb6f2745b-dirty (2025-02-05-09:32)
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
srst_only separate srst_nogate srst_open_drain connect_deassert_srst
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : STLINK V2J40M27 (API v2) VID:PID 0483:3748
Info : Target voltage: 2.877745
Info : Unable to match requested speed 2000 kHz, using 1800 kHz
Info : Unable to match requested speed 2000 kHz, using 1800 kHz
Info : clock speed 1800 kHz
Info : stlink_dap_op_connect(connect)
Info : SWD DPIDR 0x2ba01477
Info : [stm32f4x.cpu] Cortex-M4 r0p1 processor detected
Info : [stm32f4x.cpu] target has 6 breakpoints, 4 watchpoints
Info : [stm32f4x.cpu] Examination succeed
Info : [stm32f4x.cpu] starting gdb server on 3333
Info : Listening on port 3333 for gdb connections
Info : accepting 'gdb' connection on tcp/3333
[stm32f4x.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x61000000 pc: 0x0800020c msp: 0x2001ffff
Info : device id = 0x100f6413
Info : flash size = 1024 KiB
Info : flash size = 512 bytes
Warn : Prefer GDB command "target extended-remote :3333" instead of "target remote :3333"
Info : Unable to match requested speed 2000 kHz, using 1800 kHz
Info : Unable to match requested speed 2000 kHz, using 1800 kHz
[stm32f4x.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x080003e8 msp: 0x20020000
Info : Unable to match requested speed 8000 kHz, using 4000 kHz

```

1. make load
2. open new window
3. arm-none-eabi-gdb.exe
4. target remote localhost:3333
5. monitor reset init
6. monitor flash write_image erase final.elf
7. monitor reset halt
8. monitor resume

```

[info] for 1 (remote target) detached
bo@harry MINGW64 /c/Users/bo/Documents/bare_metal_embedded_system
$ arm-none-eabi-gdb.exe
GNU gdb (GNU Arm Embedded Toolchain 10.3-2021.10) 10.2.90.20210621-git
Copyright (C) 2021 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=i686-w64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

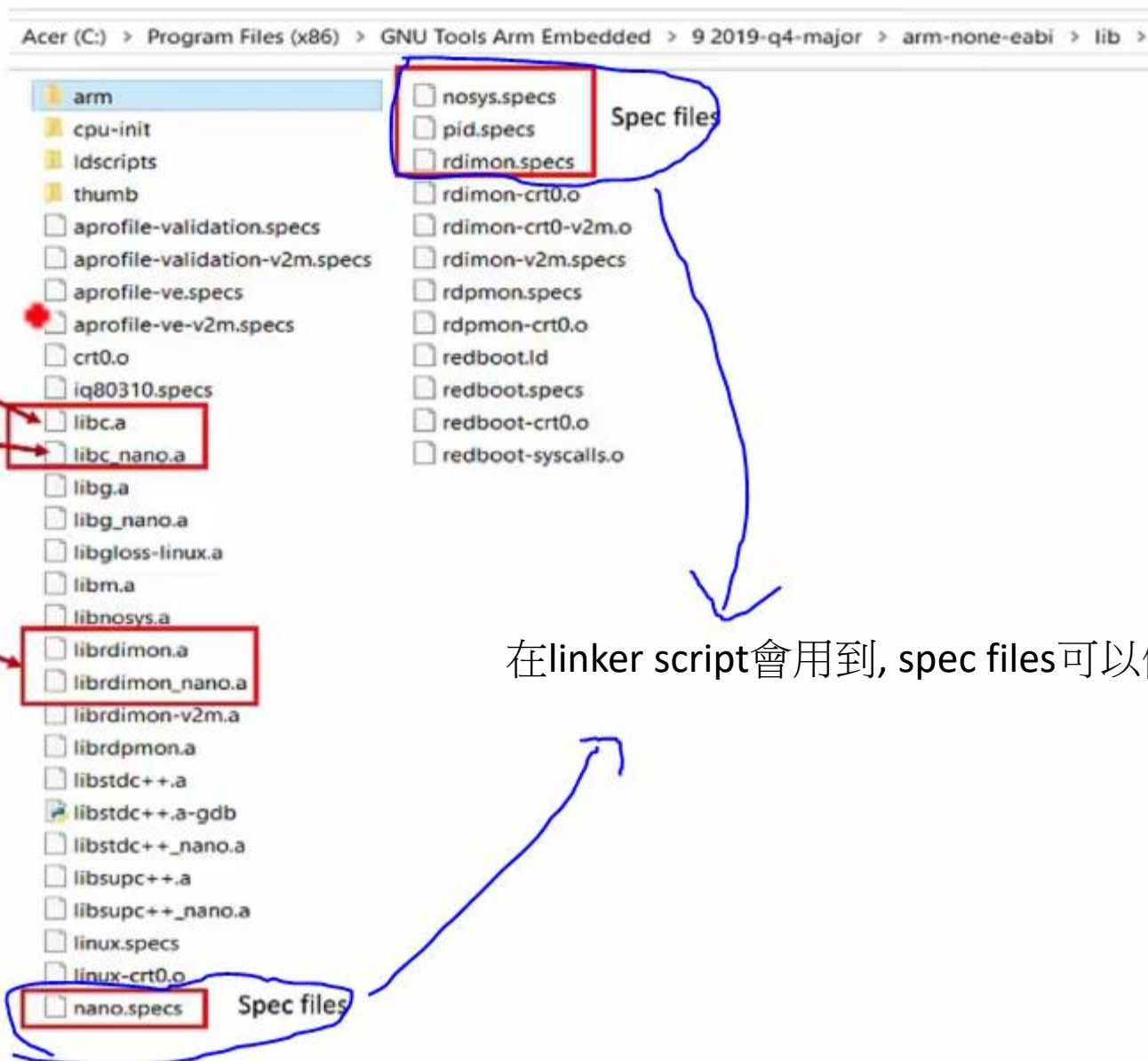
```

```

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote localhost:3333
Remote debugging using localhost:3333
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x080001b8 in ?? ()
(gdb) monitor reset init
Unable to match requested speed 2000 kHz, using 1800 kHz
Unable to match requested speed 2000 kHz, using 1800 kHz
[stm32f4x.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x08000756 msp: 00000000
Unable to match requested speed 8000 kHz, using 4000 kHz
Unable to match requested speed 8000 kHz, using 4000 kHz
(gdb) monitor flash write_image erase final.elf
auto erase enabled
wrote 16384 bytes from file final.elf in 0.574155s (27.867 KiB/s)
(gdb) monitor reset halt
Unable to match requested speed 2000 kHz, using 1800 kHz
Unable to match requested speed 2000 kHz, using 1800 kHz
[stm32f4x.cpu] halted due to breakpoint, current mode: Thread
xPSR: 0x01000000 pc: 0x08000756 msp: 00000000
(gdb) monitor resume
(gdb) |

```

Locating newlib and newlib nano



Newlib 'C' standard library

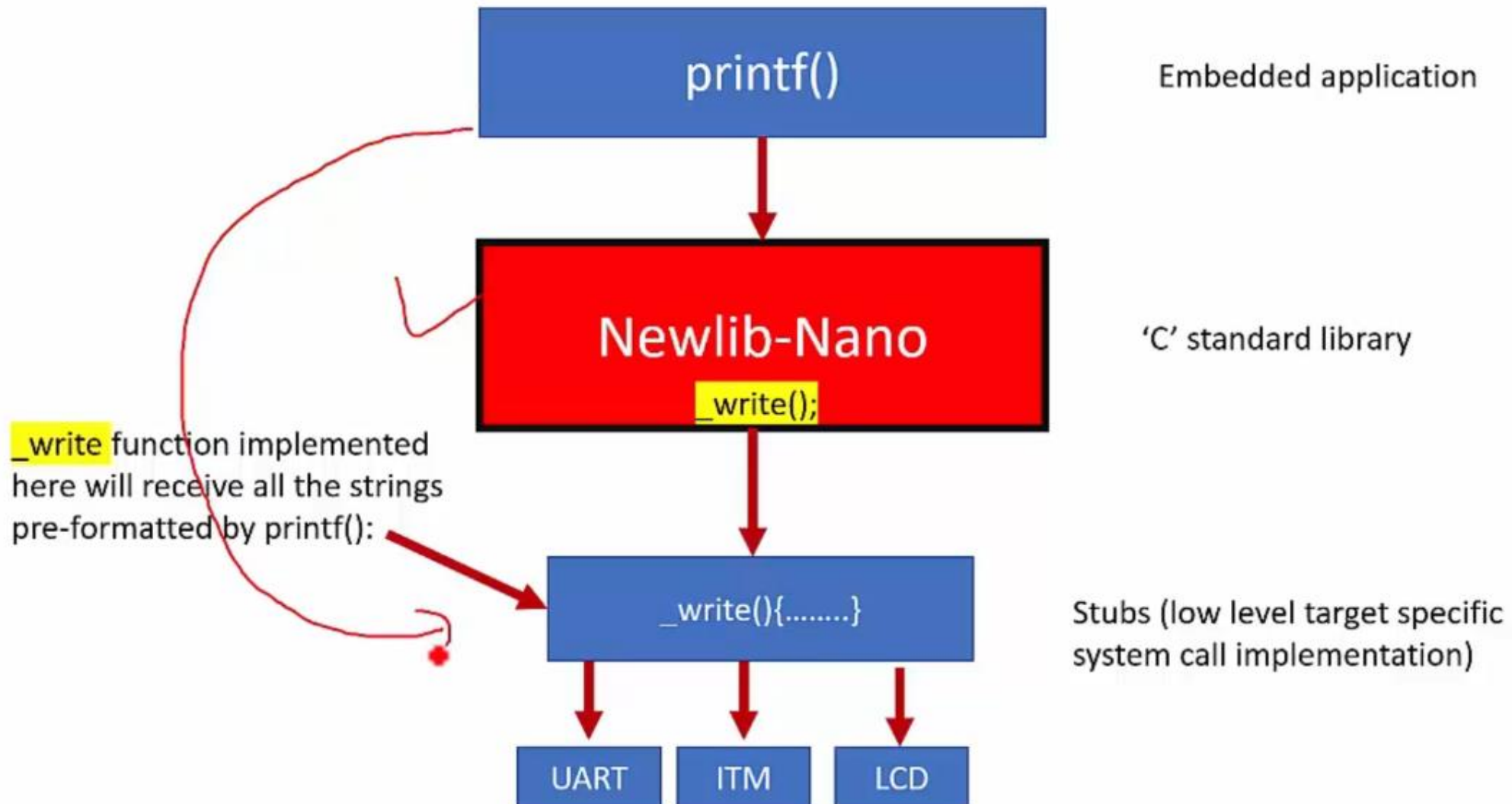
Newlib-nano 'C' standard library

semihosting library that is used for console IO operations like printf

在linker script會用到, spec files可以使code連結到libc.a or libc_nano.a

Add C standard library

1. Libc_nano.a file is Newlib-Nano
2. 要自己寫_write是用何種週邊實現(UART, ITM, LCD)



```
SECTIONS
```

```
{
```

```
    .text :
```

```
{
```

```
    * (.isr_vector)
```

```
    * (.text)
```

```
    * (.text.*)
```

合并所有.text.name1, .text.name2,

```
    * (.rodata) I
```

```
    . = ALIGN(4);
```

```
    _etext = .;
```

```
}> FLASH
```

```
.data :
```